

记整段排列的小根笛卡尔树为 T ，结点编号仍为原下标， h_u 表示 u 的子树高度，约定 $h_0 = 0$ 。

笛卡尔树的中序遍历就是原序列，因此

$$m = \text{LCA}(l, r)$$

是区间 $[l, r]$ 的根。左半部分是 lc_m 子树的一段后缀，右半部分是 rc_m 子树的一段前缀。

设 $\text{Pref}(u, x)$ 表示 u 子树中从最左端到 x 这一段的高度，初值为

$$t = 1 + h_{lc_x}.$$

从 x 向 u 上跳。若当前点是父亲 p 的右儿子，还要加入 p 及其左子树，此时

$$t \leftarrow \max(1 + h_{lc_p}, t + 1).$$

否则无需修改。后缀完全对称，初值为 $1 + h_{rc_x}$ ；当前点是父亲 p 的左儿子时，执行

$$t \leftarrow \max(1 + h_{rc_p}, t + 1).$$

所有转移都形如

$$f(t) = \max(A, t + B).$$

用二元组 (A, B) 表示。若 $\text{merge}(f, g)$ 表示 $f \circ g$ ，则

$$\text{merge}((A, B), (C, D)) = (\max(A, C + B), B + D).$$

倍增维护每段祖先路径上的函数复合。从后代向祖先查询时，新取出的上方路径放在已有复合函数的外层。

最后令

$$L = \begin{cases} \text{Suff}(lc_m, l), & l < m, \\ 0, & l = m, \end{cases} \quad R = \begin{cases} \text{Pref}(rc_m, r), & m < r, \\ 0, & m = r, \end{cases}$$

答案为

$$1 + \max(L, R).$$

时间复杂度 $O(n \log n)$ 。